

Van Loan's Method for the Active-Diffusion Covariance

Why the transition covariance is a block of a matrix exponential,
and why the closed form had to go

Abstract

The forward process of the active diffusion model requires the transition covariance $M(t) = \int_0^t e^{As} Q e^{A^\top s} ds$. This note derives Van Loan's identity, which obtains $M(t)$ as an off-diagonal block of a single 4×4 matrix exponential, explains why the previously used hand-derived closed form failed — catastrophically at $k\tau = 1$ and by up to 241% at small t elsewhere — and records the numerical verification. The essential observation is that $M(t)$ never was an integral that needed evaluating: it is the solution of a linear ODE, and augmenting the state absorbs the forcing term into a homogeneous system whose solution is an exponential.

1 The quantity we need

The active model evolves image pixels x coupled to a persistent noise η ,

$$dx = (-kx + \eta) dt + \sqrt{2T_p} dW_x, \quad (1)$$

$$d\eta = -\frac{\eta}{\tau} dt + \frac{\sqrt{2T_a}}{\tau} dW_\eta, \quad (2)$$

which in vector form $z = (x, \eta)^\top$ reads $dz = Az dt + G dW$ with

$$A = \begin{pmatrix} -k & 1 \\ 0 & -w \end{pmatrix}, \quad Q = GG^\top = \text{diag}\left(2T_p, \frac{2T_a}{\tau^2}\right), \quad w \equiv \frac{1}{\tau}. \quad (3)$$

Because the drift is linear, $z_t | z_0$ is exactly Gaussian,

$$z_t | z_0 \sim \mathcal{N}(e^{At} z_0, M(t)), \quad M(t) = \int_0^t e^{As} Q e^{A^\top s} ds. \quad (4)$$

Every part of the pipeline depends on M :

1. **Forward noising.** A Cholesky factor of $M(t)$ converts white noise into the correlated pair (d_x, d_η) added to the mean. This is the training data.
2. **The loss.** The denoising target is built from M_{11}, M_{12}, M_{22} and $\det M$; see the companion loss note.
3. **The Tweedie correction** at the end of sampling, which evaluates $M(t_\varepsilon)$ at the smallest time on the grid.

An error in M therefore corrupts the data, the labels, and the final denoising step simultaneously.

2 The closed form, and its two failure modes

The integral (4) can be evaluated by hand. Writing $a = e^{-kt}$, $b = e^{-wt}$, $T_x = T_p$, $T_y = T_a/\tau^2$, the result previously used in the code was

$$M_{11} = \frac{T_x}{k} (1 - a^2) + \frac{T_y}{k} \left[\frac{1}{w(k+w)} + \frac{4abk}{(k+w)(k-w)^2} - \frac{kb^2 + wa^2}{w(k-w)^2} \right], \quad (5)$$

$$M_{12} = \frac{T_y}{w(k^2 - w^2)} \left[k(1 - b^2) - w(1 + b^2 - 2ab) \right], \quad (6)$$

$$M_{22} = \frac{T_y}{w} (1 - b^2). \quad (7)$$

These expressions are algebraically correct and numerically unusable.

2.1 Failure 1: the removable singularity at $k\tau = 1$

When $k\tau = 1$ we have $w = k$, so $(k - w)^2 = 0$ in (5) and $k^2 - w^2 = 0$ in (6). Evaluating either expression raises a division by zero. The limit exists — the singularity is *removable* — but the formula cannot reach it.

The underlying reason is structural. At $k\tau = 1$ the matrix A becomes *defective*: it has the repeated eigenvalue $-k$ with only one eigenvector, i.e. a single 2×2 Jordan block,

$$A = -kI + N, \quad N = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad N^2 = 0 \quad \implies \quad e^{At} = e^{-kt} \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix}. \quad (8)$$

Any closed form obtained by diagonalising A carries $1/(\lambda_1 - \lambda_2)$ factors and *must* blow up there. This is why the $k\tau = 1$ arm of the τ sweep ($\tau = 0.25$ at $k = 4$) could not be run at all before the change.

2.2 Failure 2: catastrophic cancellation at small t

Away from the singularity the formula still fails, more insidiously. As $t \rightarrow 0$, $M_{11} \rightarrow 2T_p t$, which is minute; but (5) assembles it from terms of order $T_y/[k(k-w)^2]$. At $\tau = 0.2$, $k = 4$ these individual terms are

$$\frac{T_y}{k} \cdot \frac{4abk}{(k+w)(k-w)^2} \approx 71, \quad \frac{T_y}{k} \cdot \frac{kb^2 + wa^2}{w(k-w)^2} \approx 72, \quad (9)$$

whose difference must yield a contribution of order 10^{-8} . That is nine to ten orders of cancellation; `float32` carries about seven decimal digits, so nothing survives.

Table 1 gives the measured relative error of (5) in `float32` against the exact value, for the parameters actually used ($k = 4$, $T_a = 6.4$, $T_p = 10^{-3}$).

The $+10^{-8}$ constants added to M_{11} and M_{22} in the original implementation were an attempt to suppress the symptom.

τ	$k - 1/\tau$	$t = 10^{-3}$	10^{-2}	0.05	0.2	1.0	2.0
0.15	-2.67	2.5×10^{-1}	3.1×10^{-3}	3.3×10^{-5}	2.2×10^{-6}	7.5×10^{-7}	7.4×10^{-7}
0.20	-1.00	2.4×10^0	6.9×10^{-3}	9.0×10^{-4}	1.7×10^{-6}	4.5×10^{-7}	4.9×10^{-7}
0.25	0	<i>division by zero</i>					
0.40	+1.50	5.5×10^{-1}	6.0×10^{-3}	9.8×10^{-5}	2.9×10^{-6}	1.1×10^{-7}	9.9×10^{-8}
0.50	+2.00	2.3×10^{-2}	4.2×10^{-3}	6.5×10^{-5}	1.0×10^{-7}	1.8×10^{-7}	1.5×10^{-7}
1.00	+3.00	5.9×10^{-3}	2.9×10^{-4}	4.1×10^{-5}	3.1×10^{-6}	1.7×10^{-7}	1.3×10^{-7}

Table 1: Relative error of the closed form (5) in `float32`. The error grows as $|k - 1/\tau| \rightarrow 0$. At $\tau = 0.2$ it exceeds 100%: the returned M_{11} bears no relation to the true value, and the resulting matrix can fail to be positive definite, which is precisely how the Cholesky factorisation crashed. Damage is confined to $t \lesssim 0.05$ — which is exactly where fine image detail is set.

3 The trick, first in one dimension

Consider the scalar inhomogeneous linear ODE

$$\dot{y} = ay + q, \quad y(0) = 0 \quad \implies \quad y(t) = \frac{q(e^{at} - 1)}{a}. \quad (10)$$

Now exponentiate the augmented matrix obtained by appending the constant q as an extra column with a zero row:

$$\exp \left[t \begin{pmatrix} a & q \\ 0 & 0 \end{pmatrix} \right] = \begin{pmatrix} e^{at} & \frac{q(e^{at} - 1)}{a} \\ 0 & 1 \end{pmatrix}. \quad (11)$$

The solution sits in the top-right entry. Nothing subtle occurred: appending a constant row converts an *inhomogeneous* linear ODE into a *homogeneous* one, and homogeneous linear ODEs with constant coefficients are exactly what matrix exponentials solve. The forcing term has been absorbed into the generator.

Van Loan’s identity is this same manoeuvre for matrices.

4 Derivation

4.1 M satisfies a Lyapunov ODE

Differentiating (4) with Leibniz’ rule,

$$\dot{M} = AM + MA^\top + Q, \quad M(0) = 0. \quad (12)$$

This has the same shape as the scalar case: linear in the unknown, plus a constant forcing Q . So we apply the same augmentation.

4.2 The block generator

Define the 4×4 matrix and its exponential,

$$C = \begin{pmatrix} -A & Q \\ 0 & A^\top \end{pmatrix}, \quad E(t) = e^{Ct}. \quad (13)$$

Since C is block upper triangular, so is $E(t)$, and the diagonal blocks exponentiate independently:

$$E(t) = \begin{pmatrix} E_{11}(t) & E_{12}(t) \\ 0 & E_{22}(t) \end{pmatrix} = \begin{pmatrix} e^{-At} & E_{12}(t) \\ 0 & e^{A^\top t} \end{pmatrix}. \quad (14)$$

4.3 The off-diagonal block

Read the (1, 2) block off $\dot{E} = CE$:

$$\dot{E}_{12} = -A E_{12} + Q e^{A^\top t}, \quad E_{12}(0) = 0. \quad (15)$$

Multiply by the integrating factor e^{At} :

$$e^{At} \dot{E}_{12} + e^{At} A E_{12} = \frac{d}{dt} (e^{At} E_{12}) = e^{At} Q e^{A^\top t}. \quad (16)$$

The left-hand side is now an exact derivative, and the right-hand side is precisely the integrand of (4). Integrating from 0 to t and using $E_{12}(0) = 0$,

$$e^{At} E_{12}(t) = \int_0^t e^{As} Q e^{A^\top s} ds = M(t). \quad (17)$$

Finally $E_{22}(t) = e^{A^\top t}$, so $E_{22}(t)^\top = e^{At}$, giving

$$\boxed{M(t) = E_{22}(t)^\top E_{12}(t)} \quad (18)$$

where E_{12} and E_{22} are the top-right and bottom-right 2×2 blocks of e^{Ct} . One matrix exponential; no denominators; no case analysis.

The interpretive point: the integral was never something that had to be *evaluated*. It is a block of an exponential, because its integrand solves a linear ODE. The closed form (5)–(7) was one particular way of unpacking that block by hand, and a numerically poor one.

5 Implementation

```
def compute_covariance(self, t):
    k = float(self.k); w = 1.0 / float(self.tau)
    Tx = float(self.Tp); Ty = float(self.Ta) / float(self.tau) ** 2
    A = torch.tensor([[ -k, 1.0], [0.0, -w]], dtype=torch.float64)
    Q = torch.tensor([[2*Tx, 0.0], [0.0, 2*Ty]],
                     dtype=torch.float64)
    Z = torch.zeros(2, 2, dtype=torch.float64)
    C = torch.cat([torch.cat([-A, Q], 1), torch.cat([Z, A.T], 1)], 0)
    E = torch.linalg.matrix_exp(C * t.double().reshape(-1, 1, 1))
    cov = E[:, 2:, 2:].transpose(-1, -2) @ E[:, :2, 2:] # E22^T E12
    cov = 0.5 * (cov + cov.transpose(-1, -2)) # symmetrise
    return cov.to(t.dtype)
```

The assembly is done in `float64` and cast down once at the end. The explicit symmetrisation removes the $O(10^{-16})$ asymmetry left by finite precision, which matters only because Cholesky rejects non-symmetric input.

5.1 Why this is numerically stable

- `matrix_exp` uses scaling-and-squaring with Padé approximants: it evaluates $e^{Ct/2^n}$ from a series whose terms are of comparable magnitude, then squares n times. The differences of nearly-equal exponentials that destroy (5) never form.
- It never diagonalises A , so the defective case (8) needs no special handling — $e^{At} = e^{-kt}(I + Nt)$ emerges automatically. The $k\tau = 1$ point is not special to the algorithm.
- Nothing in (13) involves $k - w$, $k + w$, or any other denominator that can vanish.

5.2 Cost

One 4×4 `matrix_exp` per batch element per step, in `float64`. It is evaluated per time value, not per pixel, so at batch 128 it is negligible beside the UNet forward pass; measured epoch time was unchanged by the switch.

6 Numerical verification

Two checks, both against the shipped implementation.

The identity itself. Comparing $E_{22}^\top E_{12}$ against direct quadrature of (4) on a 4×10^5 -point grid:

τ	t	$\max \text{Van Loan} - \text{quadrature} $	M_{11}
0.25	0.05	7.1×10^{-13}	6.423485×10^{-3}
0.25	0.5	2.1×10^{-10}	6.097628×10^{-1}
0.25	2.0	3.4×10^{-9}	8.002369×10^{-1}
0.50	0.05	4.5×10^{-14}	1.789678×10^{-3}
0.50	2.0	4.3×10^{-10}	5.325359×10^{-1}

The residual is the trapezoid rule’s discretisation error, not Van Loan’s. Note that $\tau = 0.25$ is included: the closed form does not exist there, but the block exponential is unremarkable.

Structure. At $\tau = 0.25$, $t = 0.7$: $E_{11} = e^{-At}$, $E_{22} = e^{A^\top t}$ and the lower-left block vanishes, all to `allclose` tolerance — confirming (14).

7 Companion fixes

The same defect appears in the *mean*. The (1,2) entry of e^{At} is

$$b(t) = \int_0^t e^{-k(t-s)} e^{-ws} ds = e^{-kt} \frac{e^{dt} - 1}{d}, \quad d \equiv k - w, \quad (19)$$

which is 0/0 at $k\tau = 1$ with limit $t e^{-kt}$, consistent with (8). Two changes were required:

- an explicit branch, `b = t*a if abs(d) < 1e-8 else a*torch.expml(d*t)/d;`

- `expm1` in place of `exp(x)-1`. For small x , $e^x - 1$ loses almost every significant digit to cancellation (at $x = 10^{-8}$ in `float32`, e^x rounds to 1 and the difference is 0), whereas `expm1` evaluates the quantity directly.

The same treatment was applied at every site where these expressions appear: the training loss, the Tweedie correction, the equilibrium transition, and both reverse-transition routines of the SSCS sampler.

8 Consequences for the recorded results

Because M enters the noising, the labels and the Tweedie step, all runs trained before this change used a corrupted forward process at small t . Results measured on those checkpoints remain valid statements about *those models*, but they are not reproducible from the current code, and comparisons across the boundary are confounded with the numerical error — which varies by arm, being largest exactly where $|k - 1/\tau|$ is smallest. This is recorded per-arm in the results note.

9 Generalisation

Equation (18) makes no use of the specific A and Q in (3). It holds for any linear SDE in any dimension. Adding a second persistent mode, giving k matrix structure, or coupling colour channels requires only writing down the new A and Q ; `compute_covariance` is unchanged. That robustness to model changes, rather than the numerics alone, is the strongest argument for the method.

References

- [1] C. F. Van Loan, *Computing integrals involving the matrix exponential*, IEEE Transactions on Automatic Control **23**(3), 395–404, 1978.
- [2] N. J. Higham, *The scaling and squaring method for the matrix exponential revisited*, SIAM Journal on Matrix Analysis and Applications **26**(4), 1179–1193, 2005.